Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский технический университет связи и информатики»
(МТУСИ)

Кафедра «Информатика»

ОТЧЁТ

по лабораторной работе № 1
**«Экосистема языка программирования C++:
конфигурирование среды разработки и сборка
кроссплатформенных приложений»**

по дисциплине «Введение в информационные технологии»

Выполнил: студент группы БПС2605
Савичева А. С.
Вариант № 13

Проверил: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....3
1 Консольное приложение в Microsoft Visual Studio....5
1.1 Создание проекта....5
1.2 Исходный код приложения....5
1.3 Сборка и запуск приложения....6
2 Сборка приложения с помощью CMake....8
2.1 Назначение и структура проекта....8
2.2 Файл CMakeLists.txt....8
2.3 Исходный код приложения....10
2.4 Конфигурирование и сборка в терминале....11
3 GUI-приложение на Qt в Microsoft Visual Studio....13
3.1 Подготовка окружения Qt....13
3.2 Создание проекта и файл сборки....13
3.3 Исходный код приложения....14
3.4 Сборка и запуск....15
4 GUI-приложение на Qt в Qt Creator....17
4.1 Создание проекта....17
4.2 Файл проекта и исходный код....17
4.3 Сборка и запуск....18
5 Работа с платформой GitVerse....20
5.1 Знакомство с платформой....20
5.2 Документация платформы....21
5.3 Создание репозитория и загрузка файлов....22
5.4 Формирование запроса на слияние....23
ЗАКЛЮЧЕНИЕ....24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....25
ПРИЛОЖЕНИЕ А. Состав проектов....26

ВВЕДЕНИЕ

Лабораторная работа посвящена изучению инструментов экосистемы языка C++ и получению практических навыков создания, сборки и запуска приложений в различных средах разработки. В работе рассматриваются консольные приложения, система сборки CMake, библиотека Qt, среда Qt Creator, а также платформа для совместной работы с исходным кодом GitVerse.

Цель работы — освоить базовый набор инструментов экосистемы C++, научиться создавать, собирать и запускать консольные и графические приложения, а также освоить приёмы работы с системой контроля версий и платформой GitVerse.

Для достижения поставленной цели необходимо решить следующие задачи:

1) создать консольное приложение в Microsoft Visual Studio;

2) освоить систему сборки CMake и собрать кроссплатформенный проект;

3) создать GUI-приложение на Qt Widgets в Microsoft Visual Studio;

4) создать GUI-приложение на Qt Widgets в Qt Creator;

5) освоить работу с платформой GitVerse: регистрацию, создание репозитория, загрузку файлов и формирование запроса на слияние;

6) оформить отчёт по результатам работы.

Работа выполняется по варианту № 13. Все приложения выводят фамилию и инициалы студента, номер группы БПС2605 и номер варианта. Проекты размещены в каталоге C:\GUI. Перечень использованного программного обеспечения приведён в таблице 1.

Таблица 1 – Программное обеспечение, использованное при выполнении работы

Компонент	Версия / параметры
Операционная система	Windows 10 Pro, версия 10.0.19045, 64-разрядная
Среда разработки	Microsoft Visual Studio 2026 Insiders 18.12
Компилятор	MSVC v145 (14.52.37025), Windows SDK
Система сборки	CMake 4.3.1 (в составе Visual Studio)
Генератор CMake	Visual Studio 18 2026, платформа x64
Библиотека Qt	Qt 6.8.3, MSVC 2022 64-bit
Среда Qt	Qt Creator 20.0.2
Платформа Git	GitVerse (https://gitverse.ru)

1 Консольное приложение в Microsoft Visual Studio

1.1 Создание проекта

Для создания консольного приложения в Microsoft Visual Studio необходимо выполнить команду «Файл → Создать → Проект» (File → New → Project) и выбрать шаблон «Консольное приложение C++» (Console App). В диалоговом окне задаются имя проекта HelloConsole, каталог размещения C:\GUI, а также флажок «Поместить решение и проект в одном каталоге».

После создания проекта в обозревателе решений (Solution Explorer) отображаются файлы решения HelloConsole.sln, проекта HelloConsole.vcxproj и исходный файл main.cpp. В панели конфигураций устанавливаются конфигурация Debug и платформа x64. Проект автоматически содержит точку входа main(), которую необходимо заменить на код приложения.

1.2 Исходный код приложения

Исходный код приложения приведён в листинге 1.

Листинг 1 – Исходный код файла main.cpp проекта HelloConsole

```
// ============================================================
// Лабораторная работа №1
// Консольное приложение "Hello, Экосистема C++"
// ============================================================
#include <iostream>
#include <clocale>
#include <windows.h>

int main()
{
    SetConsoleOutputCP(CP_UTF8);
    setlocale(LC_ALL, ".UTF-8");

    std::cout << "Hello, Экосистема C++!" << std::endl;
    std::cout << "Компилятор: MSVC " << _MSC_VER << std::endl;
    std::cout << "Стандарт C++: " << __cplusplus << std::endl;
    std::cout << "Студент: Савичева А.С., группа БПС2605, вариант 13" << std::endl;

    system("PAUSE");
    return 0;
}
```

Директива #include <windows.h> подключает функцию SetConsoleOutputCP(), а #include <clocale> — функцию setlocale(). Совместное использование SetConsoleOutputCP(CP_UTF8) и setlocale(LC_ALL, ".UTF-8") обеспечивает корректный вывод кириллицы в

консоли Windows: первая функция переключает кодовую страницу консоли на UTF-8, вторая настраивает локализацию стандартной библиотеки C++.

Чтобы строковые литералы с кириллицей не искажались при компиляции, в свойствах проекта (C/C++ → Командная строка) добавлен параметр /utf-8, указывающий компилятору, что исходный файл сохранён в UTF-8. Макросы _MSC_VER и __cplusplus позволяют вывести версию компилятора и поддерживаемый стандарт языка. Вызов system("PAUSE") оставляет окно консоли открытым до нажатия клавиши.

На рисунке 1 показан исходный код проекта в редакторе Visual Studio.

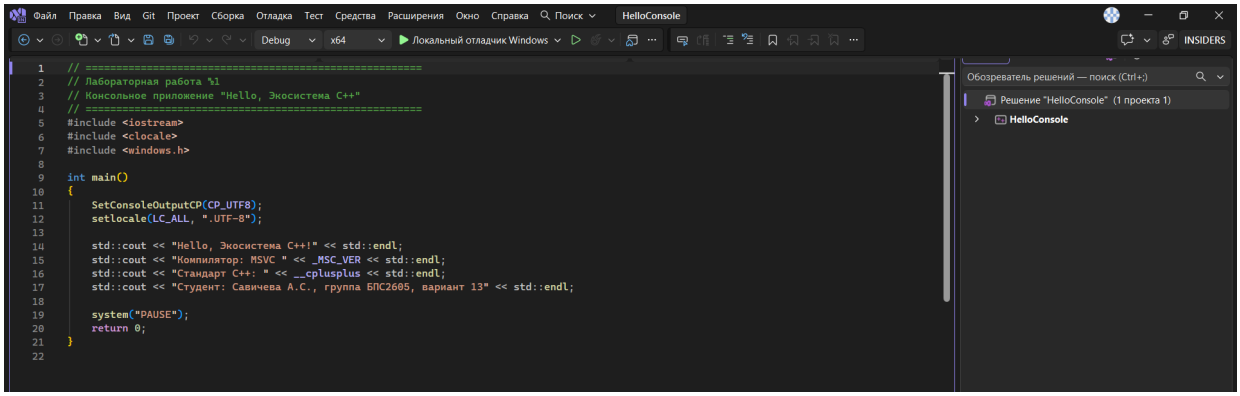


Рисунок 1 – Исходный код проекта HelloConsole в редакторе Visual Studio

1.3 Сборка и запуск приложения

Сборка и запуск выполняются командами «Сборка → Сборка решения» (Ctrl+Shift+B) и «Отладка → Запуск без отладки» (Ctrl+F5). Результат работы приложения приведён на рисунке 2.

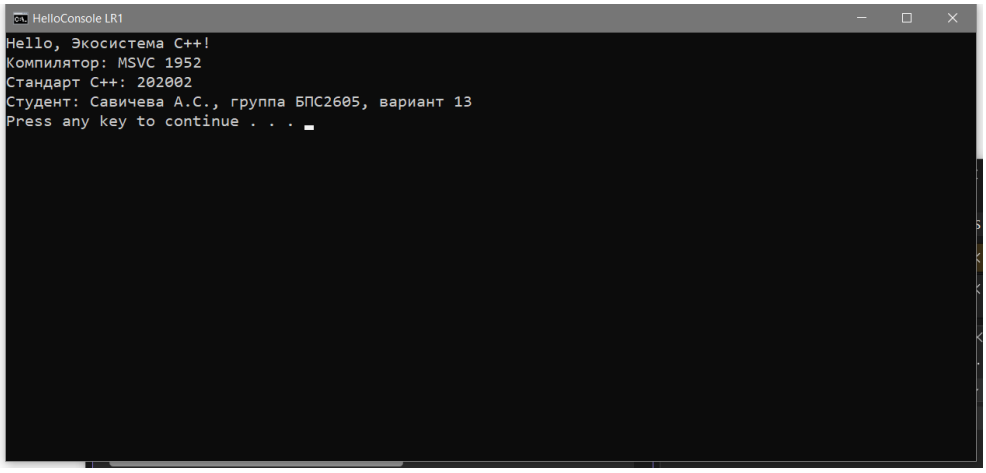


Рисунок 2 – Результат выполнения консольного приложения HelloConsole

Приложение выводит приветствие, версию компилятора MSVC 1952 (что соответствует версии 14.52), значение макроса __cplusplus 202002 (стандарт C++20), а также сведения о студенте и варианте. Кириллица отображается корректно, что подтверждает правильность настроек кодировки.

2 Сборка приложения с помощью CMake

2.1 Назначение и структура проекта

CMake — кроссплатформенная система сборки, которая по описанию проекта в файле CMakeLists.txt генерирует файлы сборки для выбранного генератора (Makefile, Ninja, Visual Studio и др.). В отличие от проекта Visual Studio, файл CMakeLists.txt не зависит от конкретной среды разработки, что позволяет собирать один и тот же проект в Windows, Linux и macOS.

Проект HelloCMake расположен в каталоге C:\GUI\HelloCMake и содержит три файла: CMakeLists.txt (описание проекта), HelloCMake.h (объявление функции studentInfo()) и HelloCMake.cpp (реализация функции и точка входа main()). Сборка выполняется вне исходного каталога — в подкаталоге build.

2.2 Файл CMakeLists.txt

Файл CMakeLists.txt проекта приведён в листинге 2. Он содержит девять логических разделов: минимальная версия CMake; объявление проекта; стандарт языка C++; определение исполняемого файла; подключение заголовочных файлов; кроссплатформенные определения препроцессора; флаги компилятора; выходной каталог; вывод итоговой информации. Каждый раздел снабжён комментариями, поэтому файл одновременно является учебным примером.

Листинг 2 – Файл CMakeLists.txt проекта HelloCMake

```
# ============================================================
# CMakeLists.txt
# Лабораторная работа №1
# Проект: HelloCMake
# Демонстрация основ CMake и кроссплатформенной сборки
# ============================================================

# ------------------------------------------------------------
# 1. МИНИМАЛЬНАЯ ВЕРСИЯ CMAKE
# ------------------------------------------------------------
# Указывает минимально необходимую версию CMake для сборки.
# Если у пользователя более старая версия — сборка прервётся
# с понятным сообщением об ошибке.
cmake_minimum_required(VERSION 3.8)

# ------------------------------------------------------------
```

```
# 2. ОБЪЯВЛЕНИЕ ПРОЕКТА
# ------------------------------------------------------------
# project() — обязательная команда. Задаёт:
#   - имя проекта (HelloCMake);
#   - версию (1.0);
#   - языки программирования (CXX — C++).
# VERSION позволяет использовать переменные
#   PROJECT_VERSION, PROJECT_VERSION_MAJOR, ... в коде.
# LANGUAGES указывает, какие языки используются. Это влияет
#   на проверку наличия соответствующих компиляторов.
project(HelloCMake
        VERSION 1.0
        DESCRIPTION "Демонстрационное приложение для ЛР №1"
        LANGUAGES CXX)

# ------------------------------------------------------------
# 3. СТАНДАРТ ЯЗЫКА C++
# ------------------------------------------------------------
# CXX_STANDARD — устанавливает стандарт C++ для всего проекта.
# CXX_STANDARD_REQUIRED — если ON, CMake требует от компилятора
#   полной поддержки указанного стандарта. При отсутствии —
#   сборка завершится с ошибкой.
# CXX_EXTENSIONS — отключает расширения компилятора (например,
#   специфичные для MSVC или GCC). Делает код более
#   переносимым между платформами.
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

# ------------------------------------------------------------
# 4. ОПРЕДЕЛЕНИЕ ЦЕЛЕВОГО ИСПОЛНЯЕМОГО ФАЙЛА
# ------------------------------------------------------------
# add_executable() — создаёт цель (target) — исполняемый файл.
# Первый аргумент: имя цели (HelloCMake).
# Далее: список исходных файлов (.cpp, .h).
# CMake автоматически определит зависимости по #include.
add_executable(HelloCMake
               "HelloCMake.cpp"
               "HelloCMake.h")

# ------------------------------------------------------------
# 5. ВКЛЮЧЕНИЕ ЗАГОЛОВОЧНЫХ ФАЙЛОВ В ЦЕЛЬ
# ------------------------------------------------------------
# target_include_directories() — указывает каталоги,
#   в которых компилятор будет искать заголовочные файлы.
# PRIVATE — директория доступна только самой цели.
# ${CMAKE_CURRENT_SOURCE_DIR} — переменная CMake, содержащая
#   абсолютный путь к каталогу с текущим CMakeLists.txt.
target_include_directories(HelloCMake PRIVATE ${CMAKE_CURRENT_SOURCE_DIR})

# ------------------------------------------------------------
# 6. КРОССПЛАТФОРМЕННЫЕ ОПРЕДЕЛЕНИЯ ПРЕПРОЦЕССОРА
# ------------------------------------------------------------
# target_compile_definitions() — добавляет макросы препроцессора
#   для конкретной цели.
# В зависимости от операционной системы определяется разный
#   макрос, что позволяет писать платформозависимый код.
if(WIN32)
  target_compile_definitions(HelloCMake PRIVATE PLATFORM_WINDOWS)
elseif(UNIX AND NOT APPLE)
  target_compile_definitions(HelloCMake PRIVATE PLATFORM_LINUX)
elseif(APPLE)
  target_compile_definitions(HelloCMake PRIVATE PLATFORM_MACOS)
endif()

# ------------------------------------------------------------
# 7. КОМПИЛЯТОР-СПЕЦИФИЧНЫЕ ФЛАГИ
# ------------------------------------------------------------
# Для MSVC: установка уровня предупреждений /W4 и кодировки
#   исходных текстов UTF-8 (/utf-8), чтобы кириллица в
#   строковых литералах не искажалась.
```

```
if(MSVC)
  target_compile_options(HelloCMake PRIVATE /W4 /utf-8 /Zc:__cplusplus)
endif()
# Для GCC/Clang: флаги -Wall -Wextra -Wpedantic
if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
  target_compile_options(HelloCMake PRIVATE -Wall -Wextra -Wpedantic)
endif()

# ------------------------------------------------------------
# 8. УСТАНОВКА ВЫХОДНОГО КАТАЛОГА
# ------------------------------------------------------------
# По умолчанию CMake размещает исполняемые файлы в сложной
# иерархии (в зависимости от конфигурации). Свойства
# RUNTIME_OUTPUT_DIRECTORY задают единый выходной каталог
# bin/ для всех конфигураций (в том числе для многоконфигура-
# ционного генератора Visual Studio).
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
set_target_properties(HelloCMake PROPERTIES
    RUNTIME_OUTPUT_DIRECTORY         ${CMAKE_BINARY_DIR}/bin
    RUNTIME_OUTPUT_DIRECTORY_DEBUG   ${CMAKE_BINARY_DIR}/bin
    RUNTIME_OUTPUT_DIRECTORY_RELEASE ${CMAKE_BINARY_DIR}/bin)

# ------------------------------------------------------------
# 9. ВЫВОД ИТОГОВОЙ ИНФОРМАЦИИ ПРИ КОНФИГУРАЦИИ
# ------------------------------------------------------------
# message() — выводит сообщения при запуске cmake.
# STATUS — информационное сообщение (не ошибка).
message(STATUS "========================================")
message(STATUS "Проект:                ${PROJECT_NAME}")
message(STATUS "Версия:                ${PROJECT_VERSION}")
message(STATUS "Стандарт C++:          ${CMAKE_CXX_STANDARD}")
message(STATUS "Компилятор C++:        ${CMAKE_CXX_COMPILER_ID}")
message(STATUS "Сборочный каталог:     ${CMAKE_BINARY_DIR}")
message(STATUS "Исходный каталог:      ${CMAKE_CURRENT_SOURCE_DIR}")
message(STATUS "Выходной каталог:      ${CMAKE_RUNTIME_OUTPUT_DIRECTORY}")
message(STATUS "Платформа:             ${CMAKE_SYSTEM_NAME}")
message(STATUS "========================================")
```

В разделе 7 для компилятора MSVC заданы флаги /W4 (повышенный уровень предупреждений), /utf-8 (кодировка исходных текстов UTF-8) и /Zc:__cplusplus (корректное значение макроса __cplusplus). Для GCC и Clang используются флаги -Wall, -Wextra и -Wpedantic. Определение PLATFORM_WINDOWS позволяет выбрать платформозависимую ветку кода при компиляции.

2.3 Исходный код приложения

Заголовочный файл HelloCMake.h содержит объявление функции studentInfo() (листинг 3), а файл HelloCMake.cpp — её реализацию и функцию main() (листинг 4). Вынесение функции в отдельный модуль демонстрирует совместную компиляцию нескольких файлов проекта.

Листинг 3 – Заголовочный файл HelloCMake.h

```
// ============================================================
// Лабораторная работа №1
// Проект: HelloCMake
// Заголовочный файл приложения
// ============================================================
#pragma once

// Возвращает строку с данными студента.
// Вынесена в отдельный модуль, чтобы продемонстрировать
// совместную компиляцию нескольких файлов проекта.
const char* studentInfo();
```

Листинг 4 – Исходный код файла HelloCMake.cpp

```
// ============================================================
// Лабораторная работа №1
// Консольное приложение "Hello, Экосистема C++"
// Проект: HelloCMake (кроссплатформенная сборка CMake)
// ============================================================
#include "HelloCMake.h"

#include <iostream>
#include <clocale>
#include <windows.h>

using namespace std;

const char* studentInfo()
{
    return "Студент: Савичева А.С., группа БПС2605, вариант 13";
}

int main()
{
    SetConsoleOutputCP(CP_UTF8);
    setlocale(LC_ALL, ".UTF-8");

    cout << "Hello, Экосистема C++!" << endl;
    cout << "Компилятор: MSVC " << _MSC_VER << endl;
    cout << "Стандарт C++: " << __cplusplus << endl;
    cout << studentInfo() << endl;

#ifdef PLATFORM_WINDOWS
    cout << "Платформа: Windows" << endl;
#elif defined(PLATFORM_LINUX)
    cout << "Платформа: Linux" << endl;
#elif defined(PLATFORM_MACOS)
    cout << "Платформа: macOS" << endl;
#endif

    system("PAUSE");
    return 0;
}
```

2.4 Конфигурирование и сборка в терминале

Конфигурирование и сборка выполняются в терминале (например, в «Командной строке разработчика» или во встроенном терминале Visual Studio). Последовательность команд приведена ниже.

```
cd C:\GUI\HelloCMake
mkdir build
cd build
cmake .. -G "Visual Studio 18 2026" -A x64
cmake --build . --config Debug
bin\HelloCMake.exe
```

Первая команда cmake выполняет конфигурирование: проверяет компилятор, генерирует проект Visual Studio и выводит итоговую информацию (имя проекта, версию, стандарт C++, компилятор, каталоги, платформу). Вторая команда запускает сборку через MSBuild. Готовый исполняемый файл размещается в каталоге build\bin согласно настройкам RUNTIME_OUTPUT_DIRECTORY.

Результат конфигурирования, сборки и запуска приведён на рисунке 3.

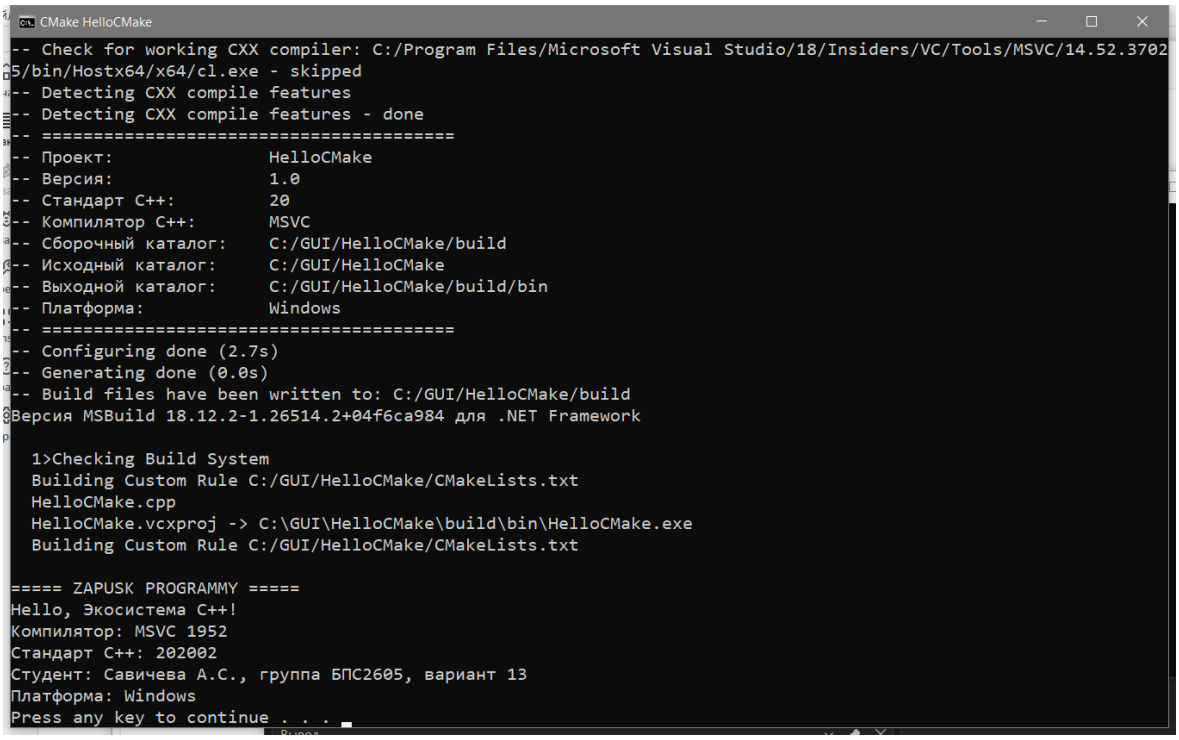


Рисунок 3 – Конфигурирование, сборка и запуск проекта HelloCMake

В выводе видно, что CMake обнаружил компилятор MSVC 14.52, стандарт C++20 и платформу Windows, сборка завершилась без ошибок, а приложение вывело те же данные, что и консольный проект Visual Studio.

3 GUI-приложение на Qt в Microsoft Visual Studio

3.1 Подготовка окружения Qt

Для разработки GUI-приложений на Qt в Visual Studio используется расширение Qt Visual Studio Tools. В работе установлен Qt 6.8.3 для MSVC 2022 64-bit в каталог C:\Qt\6.8.3\msvc2022_64. Через меню «Расширения → Управление расширениями» устанавливается Qt Visual Studio Tools, после чего в меню «Расширения → Qt VS Tools → Qt Versions» добавляется версия Qt с указанием каталога установки.

3.2 Создание проекта и файл сборки

Проект создаётся командой «Файл → Создать → Проект» с шаблоном Qt Widgets Application. В качестве имени указано HelloQt, система сборки — CMake. Среда создаёт файлы CMakeLists.txt и main.cpp. Файл CMakeLists.txt приведён в листинге 5.

Листинг 5 – Файл CMakeLists.txt проекта HelloQt

```
# ============================================================
# CMakeLists.txt
# Лабораторная работа №1
# Проект: HelloQt (GUI-приложение на Qt Widgets)
# ============================================================

cmake_minimum_required(VERSION 3.21)

project(HelloQt
        VERSION 1.0
        DESCRIPTION "GUI-приложение Qt Widgets для ЛР №1"
        LANGUAGES CXX)

# ------------------------------------------------------------
# Стандарт языка C++
# ------------------------------------------------------------
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

# ------------------------------------------------------------
# Автоматическая обработка метаобъектов Qt (moc), форм (uic)
# и ресурсов (rcc)
# ------------------------------------------------------------
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTOUIC ON)
set(CMAKE_AUTORCC ON)

# ------------------------------------------------------------
# Поиск библиотек Qt 6 (модуль Widgets)
# ------------------------------------------------------------
find_package(Qt6 REQUIRED COMPONENTS Widgets)
```

```
# ------------------------------------------------------------
# Исполняемый файл GUI-приложения
# WIN32 — приложение без консольного окна
# ------------------------------------------------------------
add_executable(HelloQt WIN32 "main.cpp")

target_link_libraries(HelloQt PRIVATE Qt6::Widgets)

# ------------------------------------------------------------
# Для MSVC: предупреждения и кодировка исходных текстов UTF-8
# ------------------------------------------------------------
if(MSVC)
  target_compile_options(HelloQt PRIVATE /W4 /utf-8)
endif()

# ------------------------------------------------------------
# Единый выходной каталог bin/ для всех конфигураций
# ------------------------------------------------------------
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)
set_target_properties(HelloQt PROPERTIES
    RUNTIME_OUTPUT_DIRECTORY         ${CMAKE_BINARY_DIR}/bin
    RUNTIME_OUTPUT_DIRECTORY_DEBUG   ${CMAKE_BINARY_DIR}/bin
    RUNTIME_OUTPUT_DIRECTORY_RELEASE ${CMAKE_BINARY_DIR}/bin)
```

Директивы CMAKE_AUTOMOC, CMAKE_AUTOUIC и CMAKE_AUTORCC включают автоматическую обработку метаобъектов, форм и ресурсов Qt. Команда find_package(Qt6 REQUIRED COMPONENTS Widgets) подключает модуль Widgets, а target_link_libraries связывает приложение с библиотекой Qt6::Widgets. Ключевое слово WIN32 в add_executable создаёт GUI-приложение без консольного окна.

3.3 Исходный код приложения

Исходный код приложения приведён в листинге 6.

Листинг 6 – Исходный код файла main.cpp проекта HelloQt

```
// ========================================================
// Лабораторная работа №1
// GUI-приложение "Hello, Qt"
// Студент: Савичева А.С., группа БПС2605, вариант 13
// ========================================================
#include <QApplication>
#include <QLabel>
#include <QString>

int main(int argc, char* argv[])
{
    QApplication app(argc, argv);

    QLabel label;
    // Исходный файл сохранён в UTF-8, поэтому строку декодируем как UTF-8
    label.setText(QString::fromUtf8("Савичева А.С., группа БПС2605\n"
                                    "Вариант 13\n"
                                    "Hello, Qt!"));
    label.setAlignment(Qt::AlignCenter);
    label.resize(400, 250);
    label.setStyleSheet("font-size: 16px; font-weight: bold;");
    label.setWindowTitle("HelloQt \u2014 Лабораторная работа №1");
    label.show();

    return app.exec();
}
```

Объект QApplication управляет циклом обработки событий приложения. Виджет QLabel отображает текст, выровненный по центру. Поскольку исходный файл сохранён в UTF-8, строка передаётся через QString::fromUtf8(), что гарантирует корректное отображение кириллицы. Метод setStyleSheet задаёт размер и начертание шрифта, а setWindowTitle — заголовок окна.

3.4 Сборка и запуск

Сборка выполняется командой «Сборка → Сборка решения», запуск — «Отладка → Запуск без отладки». Окно приложения приведено на рисунке 4.

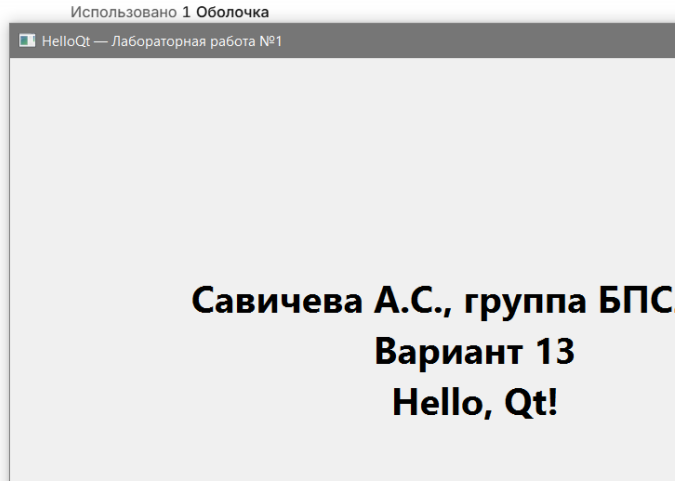


Рисунок 4 – Окно GUI-приложения HelloQt

Приложение отображает фамилию и инициалы студента, группу БПС2605, вариант 13 и приветствие «Hello, Qt!». Кириллица выводится корректно. В каталоге сборки рядом с исполняемым файлом размещены библиотеки Qt (Qt6Core.dll, Qt6Gui.dll, Qt6Widgets.dll и др.), необходимые для запуска приложения.

4 GUI-приложение на Qt в Qt Creator

4.1 Создание проекта

Qt Creator — среда разработки, входящая в состав Qt. Проект создаётся командой «Файл → Создать проект» (File → New Project), в окне шаблонов выбирается «Application (Qt) → Qt Widgets Application». В качестве имени указывается HelloQtCreator, система сборки — qmake, набор — Desktop Qt 6.8.3 MSVC2022 64bit. Остальные параметры оставляются по умолчанию.

4.2 Файл проекта и исходный код

Файл проекта HelloQtCreator.pro приведён в листинге 7, исходный код — в листинге 8.

Листинг 7 – Файл проекта HelloQtCreator.pro

```
# ========================================================
# HelloQtCreator.pro
# Лабораторная работа №1
# GUI-приложение на Qt Widgets (сборка qmake)
# Студент: Савичева А.С., группа БПС2605, вариант 13
# ========================================================

QT       += core gui widgets

CONFIG   += c++20

TARGET    = HelloQtCreator
TEMPLATE  = app

SOURCES  += main.cpp

# Кодировка исходных текстов UTF-8 для MSVC
msvc {
    QMAKE_CXXFLAGS += /utf-8
}
```

Листинг 8 – Исходный код файла main.cpp проекта HelloQtCreator

```
// ========================================================
// Лабораторная работа №1
// GUI-приложение "Hello, Qt" (Qt Creator, qmake)
// Студент: Савичева А.С., группа БПС2605, вариант 13
// ========================================================
#include <QApplication>
#include <QLabel>

int main(int argc, char* argv[])
{
    QApplication app(argc, argv);

    QLabel label(u8"Савичева А.С., группа БПС2605\n"
                 "Вариант 13\n"
                 "Hello, Qt!");
    label.setAlignment(Qt::AlignCenter);
    label.resize(400, 250);
    label.setStyleSheet("font-size: 16px; font-weight: bold;");
    label.setWindowTitle("HelloQtCreator \u2014 Лабораторная работа №1");
    label.show();

    return app.exec();
}
```

В файле .pro директива QT += core gui widgets подключает необходимые модули Qt, CONFIG += c++20 включает стандарт C++20, TEMPLATE = app задаёт тип приложения, SOURCES += main.cpp добавляет исходный файл. Для MSVC добавлен флаг /utf-8.

В отличие от проекта Visual Studio, строка с кириллицей записана с использованием литерала u8"", который гарантирует кодировку UTF-8 независимо от кодировки исходного файла. Остальной код аналогичен проекту HelloQt.

4.3 Сборка и запуск

Сборка выполняется командой «Сборка → Собрать проект» (Ctrl+B), запуск — «Сборка → Запустить» (Ctrl+R). Окно Qt Creator с открытым проектом и окно приложения приведены на рисунке 5.

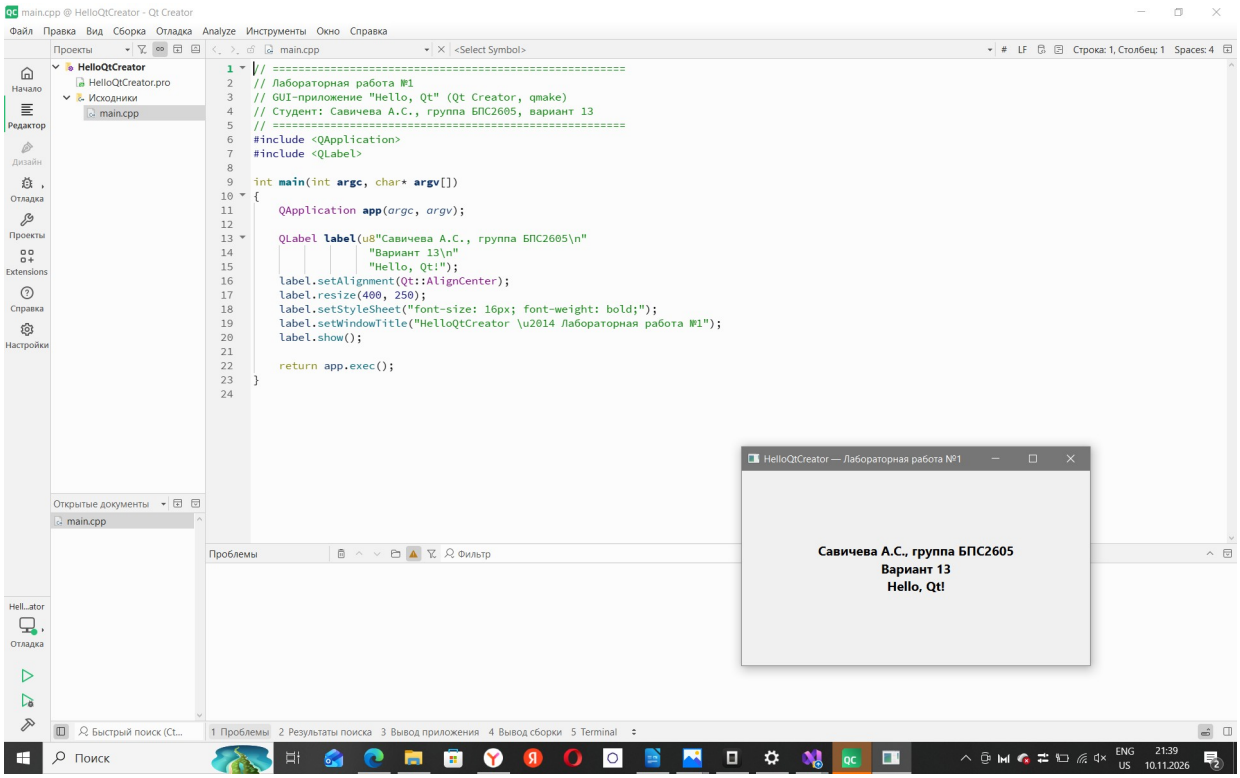


Рисунок 5 – Проект HelloQtCreator в Qt Creator и окно приложения

Приложение успешно собрано и запущено, кириллица отображается корректно. Использование qmake и Qt Creator подтверждает, что один и тот же код может собираться различными инструментами экосистемы Qt.

5 Работа с платформой GitVerse

5.1 Знакомство с платформой

GitVerse — российская платформа для хранения и совместной разработки исходного кода, предоставляющая возможности Git-репозиториев, запросов на слияние (merge request), задач, вики, средств непрерывной интеграции и доставки (CI/CD) и другие. Платформа доступна по адресу https://gitverse.ru (рисунок 6). Для входа используются единая учётная запись GigaID, а также Сбер ID и Cloud.ru ID (рисунок 7).

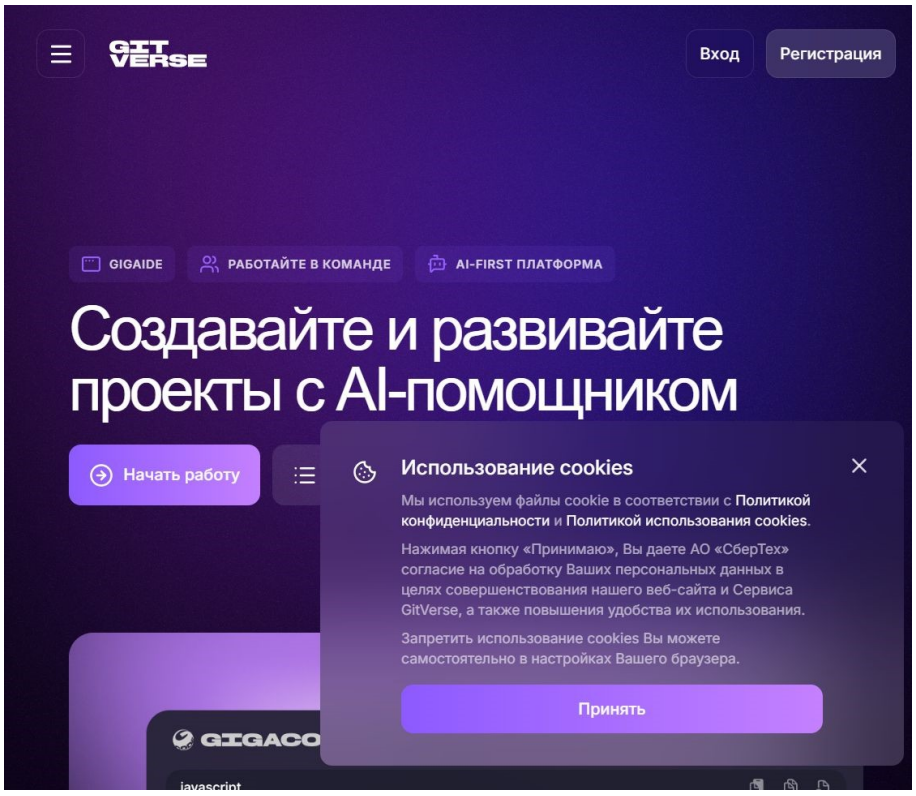


Рисунок 6 – Главная страница платформы GitVerse

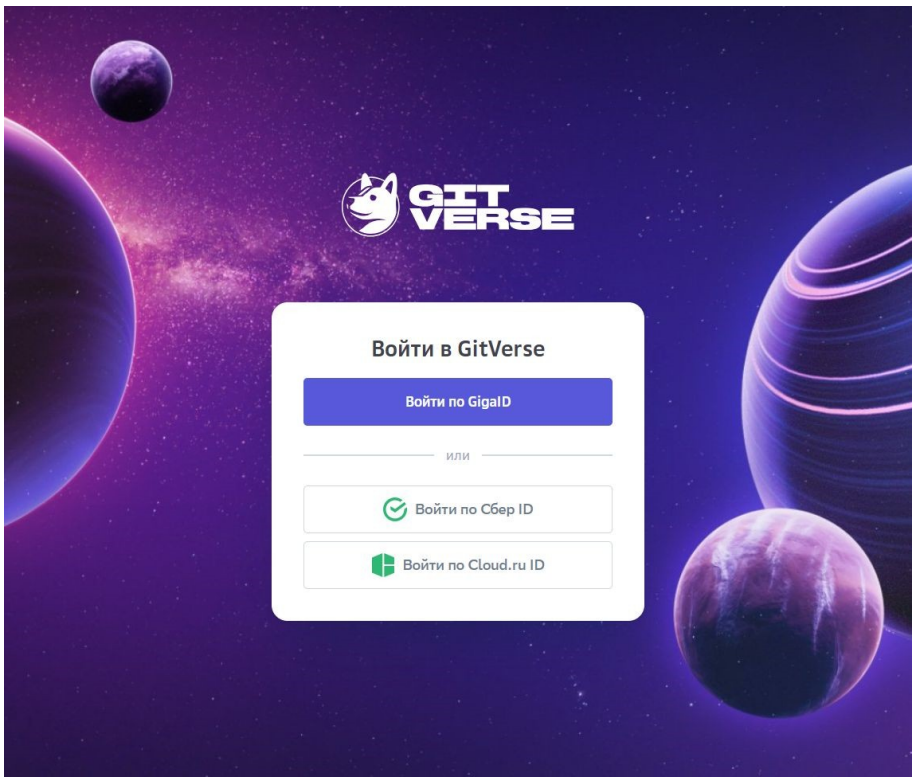


Рисунок 7 – Страница входа в GitVerse

В соответствии с заданием требуется зарегистрироваться на платформе, создать репозиторий MTUCI_Introduction_to_IT_БПС2605, загрузить файлы с результатами работы и сформировать запрос на слияние. Порядок действий приведён в пп. 5.3 и 5.4.

5.2 Документация платформы

На платформе доступна подробная документация (рисунок 8), в том числе раздел «Репозитории», описывающий создание, клонирование, импорт и настройку репозиториев. Перед выполнением работы изучены разделы «Быстрый старт» и «Репозитории».

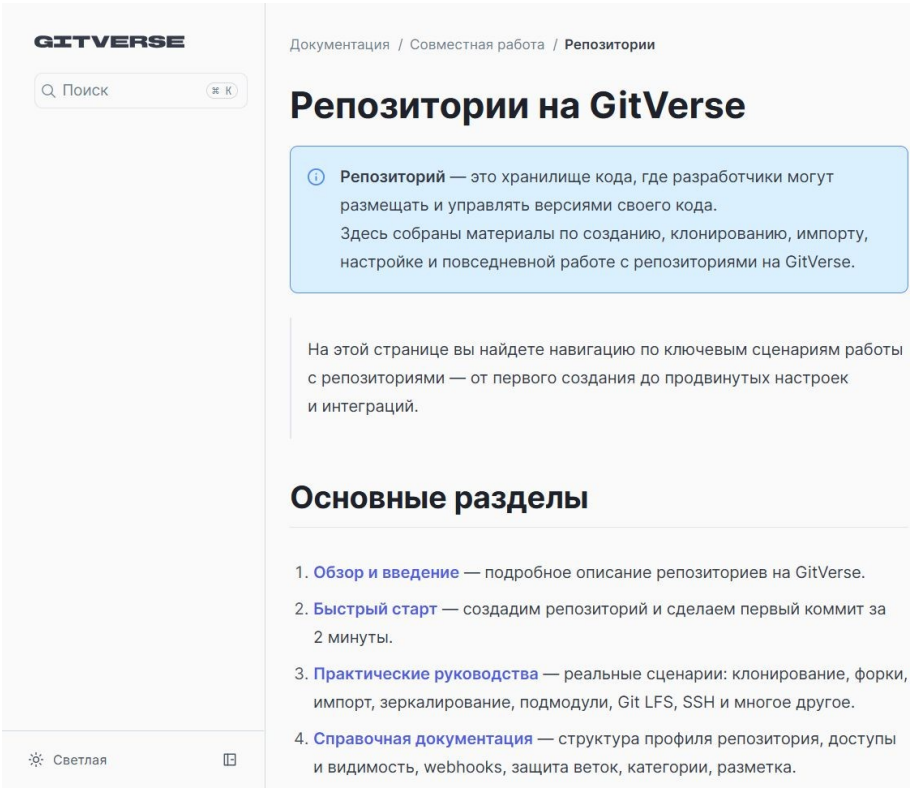


Рисунок 8 – Раздел документации GitVerse «Репозитории»

5.3 Создание репозитория и загрузка файлов

Для размещения результатов работы на платформе GitVerse необходимо выполнить следующие действия.

1. Войти на GitVerse через GigaID и заполнить данные профиля.

2. В правом верхнем углу нажать «+» и выбрать «Новый репозиторий».

3. Указать имя репозитория MTUCI_Introduction_to_IT_БПС2605, выбрать тип «Публичный» и не инициализировать его файлом README.

4. Создать ветку, например Savicheva_LR1, от основной ветки main.

5. В веб-интерфейсе в каталоге Students/БПС2605/Вариант 13 нажать «Добавить файл» → «Загрузить файлы» и разместить исходные коды проектов и отчёт.

6. Указать сообщение коммита, например «Лабораторная работа № 1, вариант 13», и сохранить изменения.

При загрузке нескольких файлов допускается перетаскивание их в окно браузера. После сохранения файлы становятся доступны в выбранной ветке репозитория.

5.4 Формирование запроса на слияние

Чтобы результаты работы попали в основную ветку репозитория, необходимо сформировать запрос на слияние (Merge Request).

1. После загрузки файлов нажать «Создать запрос на слияние».

2. Выбрать исходную ветку Savicheva_LR1 и целевую ветку main.

3. Заполнить заголовок и описание запроса, указать преподавателя-рецензента.

4. Создать запрос и дождаться его проверки и слияния.

После слияния файлы становятся доступны в основной ветке репозитория, а работа считается сданной.

Примечание. *Для выполнения п. 6 методических указаний требуется личная учётная запись GitVerse, привязанная к GigaID. Приведённые в отчёте снимки экрана (рисунки 6–8) относятся к общедоступным страницам платформы; действия пп. 5.3–5.4 выполняются в личном кабинете студента.*

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы освоены основные инструменты экосистемы C++. Создано и собрано консольное приложение в Microsoft Visual Studio; изучена система сборки CMake и выполнена сборка кроссплатформенного проекта из командной строки; созданы GUI-приложения на Qt Widgets в Visual Studio и Qt Creator; выполнено знакомство с платформой GitVerse и изучен порядок размещения исходного кода в репозитории.

Все четыре проекта успешно собираются и запускаются, кириллица во всех приложениях отображается корректно. Полученные навыки позволяют выбирать подходящий инструмент сборки и среду разработки в зависимости от задачи, а также организовывать совместную работу с исходным кодом.

Результаты работы могут быть использованы при выполнении последующих лабораторных работ, связанных с разработкой приложений на C++ и Qt.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методические указания к лабораторной работе № 1 «Экосистема языка программирования C++: конфигурирование среды разработки и сборка кроссплатформенных приложений». — М.: МТУСИ, 2026.

2. Документация Microsoft Visual Studio [Электронный ресурс]. — URL: https://learn.microsoft.com/visualstudio/ (дата обращения: 10.11.2026).

3. CMake Reference Documentation [Электронный ресурс]. — URL: https://cmake.org/documentation/ (дата обращения: 10.11.2026).

4. Qt 6 Documentation [Электронный ресурс]. — URL: https://doc.qt.io/qt-6/ (дата обращения: 10.11.2026).

5. Документация GitVerse [Электронный ресурс]. — URL: https://gitverse.ru/docs (дата обращения: 10.11.2026).

6. cppreference.com — C++ reference [Электронный ресурс]. — URL: https://en.cppreference.com/ (дата обращения: 10.11.2026).

ПРИЛОЖЕНИЕ А. Состав проектов

Перечень файлов, входящих в состав созданных проектов, приведён в таблице А.1.

Таблица А.1 – Файлы проектов

Проект	Файлы	Назначение
HelloConsole	main.cpp, HelloConsole.vcxproj, HelloConsole.sln	Консольное приложение, проект Visual Studio
HelloCMake	CMakeLists.txt, HelloCMake.h, HelloCMake.cpp	Консольное приложение, сборка CMake
HelloQt	CMakeLists.txt, main.cpp	GUI-приложение Qt Widgets, сборка CMake в Visual Studio
HelloQtCreator	HelloQtCreator.pro, main.cpp	GUI-приложение Qt Widgets, сборка qmake в Qt Creator